Ders 4 Genişletilmiş Notu: Sıralı Örüntü Madenciliği, Gözetimli Öğrenme ve K-Means Kümeleme

Ders 4 Genişletilmiş Notu: Sıralı Örüntü Madenciliği, Gözetimli Öğrenme ve K-Means Kümeleme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Planı ve Bağlamı
- 2. Sıralı Örüntü Madenciliği: Neden Sıra Önemli?
- 3. Gösterim: Dizi, Eleman, Boyut ve Uzunluk
- 4. Alt Dizi ve Üst Dizi
- 5. Problem Tanımı ve Müşteri Dizileri Örneği
- 6. GSP Algoritması ve Apriori ile Benzerliği
- 7. Tıklama Akışı Verisinde Sıralı Örüntüler
- 8. Gözetimli Öğrenme: Temel Kavramlar
- 9. Öğrenme Ne Demektir?
- 10. Eğitim ve Test Verisinin Dağılımı
- 11. Karar Ağacı
- 12. Sınıf İlişkilendirme Kurallarıyla Sınıflandırma
- 13. Naive Bayes Sınıflandırıcı
- 14. k En Yakın Komşu (kNN)
- 15. Gözetimsiz Öğrenme ve Kümeleme
- 16. Kümeleme Kalitesi
- 17. k-Means Algoritması
- 18. Durma Ölçütleri ve Uzaklık Fonksiyonu
- 19. Disk Üzerindeki Veride k-Means ve Karmaşıklık
- 20. k-Means'in Zayıf Yönleri ve Çözümleri
- 21. Kümelerin Şekli ve Algoritma Seçimi
- 22. Küme Gösterimi
- 23. Sonraki Dersler ve Proje Fikirleri
- Örnek Sorular

1. Dersin Planı ve Bağlamı

Hoca derse, bugün işlenecek konu sayısının yoğun olduğunu söyleyerek başladı. Dersin kitabı Web Data Mining kitabıdır ve dersler kitabın bölümlerini (chapter) sırayla izler. Geçen hafta ilişkilendirme kuralı madenciliği (association rule mining) işlenmişti. Arada Perşembe günü tatile denk geldiği için ders yapılmamıştı.

Bu derste üç konu vardı:

- Sıralı örüntü madenciliği (sequential pattern mining), 2. bölümün devamı.
- Gözetimli öğrenme (supervised learning), 3. bölüm.
- Vakit kaldığı ölçüde gözetimsiz öğrenme (unsupervised learning), 4. bölüm.

Hoca bu konuları neden gözden geçirdiğini de açıkladı. İlişkilendirme kuralı madenciliği, sıralı örüntü madenciliği, gözetimli ve gözetimsiz öğrenme, ileride işlenecek asıl web madenciliği

konularının ön koşuludur. Öğrencilerin bir kısmı aynı dönem veri madenciliği (data mining) dersini aldığı için bu konuları orada da görüyordu. Bu konuları ilk kez öğrenenler de olduğu için bölümler burada tekrar ediliyordu. Hoca, slaytların ve bölümlerin PDF'lerinin kitabın web sitesinde bulunduğunu hatırlattı.

2. Sıralı Örüntü Madenciliği: Neden Sıra Önemli?

Hoca önce ilişkilendirme kuralı madenciliğini hatırlattı. Orada işlem verisi (transaction data) vardı. Her işlemin bir kimliği (transaction ID) ve içinde birkaç öğe (item) bulunuyordu. Algoritma önce sık öğe kümelerini (frequent itemsets) bulur, en büyük sık öğe kümelerinden de ilişkilendirme kurallarını çıkarır. Ancak bu yaklaşım öğelerin **hangi sırayla** alındığına bakmaz. Yalnız öğelerin aynı işlemde birlikte alınıp alınmadığına bakar.

Sıralı örüntü madenciliği ise öğelerin alınış sırasını dikkate alır. Birçok uygulamada bu sıra önemlidir:

- **Market sepeti analizi:** İnsanların ürünleri hangi sırayla aldığını bulmak ilginçtir.
- **Web madenciliği:** Bizim için asıl önemli olan budur, çünkü web verisiyle, yani tıklama akışı (click stream) verisiyle çalışırız. Kullanıcının bir sitede yaptığı her tıklama bizim için bir veridir.

Hoca gezinme örüntüsünü (navigational pattern) şöyle anlattı. Kullanıcı sitenin ana sayfasından başlar. Bir bağlantıya tıklar ve aynı sitenin başka bir sayfasına geçer. Böyle devam ederek sitede bir gezinme örüntüsü izler. Eğer **sık gezinme örüntülerini** bulabilirsek, kullanıcının bir sonraki hareketini anlayabilir, yani tahmin edebiliriz. Buna göre pazarlama gibi eylemler yapılabilir. Bu yüzden işlemlerdeki öğelerin sırası önemlidir ve sıralı örüntü madenciliği algoritmaları kullanılır.

3. Gösterim: Dizi, Eleman, Boyut ve Uzunluk

Bu kategorideki algoritmaları anlamak için, ilişkilendirme kurallarındakine benzer bir gösterim kullanılır:

- I: öğelerin kümesi.
- X: bir öğe kümesi (itemset). İlişkilendirme kurallarından bildiğimiz gibi, öğelerden oluşan bir kümedir.
- **Dizi (sequence):** Elemanlardan oluşan sıralı bir listedir:

$$s = \langle a_1 a_2 \dots a_r \rangle$$

Dizinin her elemanı (element) a_i bir öğe kümesidir. Yani a_1 bir ya da daha fazla öğeden oluşur, a_2 de öyle. Hoca bunun nasıl okunacağını şöyle anlattı: Önce a_1'deki öğeler satın alınmıştır, hemen ardından a_2'deki öğeler, en sonunda da a_r'deki öğeler. Yani dizinin elemanları arasında bir sıra vardır.

Dizi için iki ölçü tanımlanır:

- **Boyut (size):** Dizideki eleman sayısıdır. $\langle a_1 a_2 \dots a_r \rangle$ dizisinde $1, 2, 3, \dots, r$ diye sayılır, boyut r'dir.
- **Uzunluk (length):** Dizideki toplam öğe sayısıdır. $a_1, a_2, \dots, a_r$ öğe kümelerinin hepsindeki öğeler birlikte sayılır. Uzunluğu k olan diziye k-dizi (k-sequence) denir.

Hocanın örneği $\langle \{3\}\{4,5\}\{8\} \rangle$ dizisiydi. Burada öğeler bir markette satılan ürünler gibi düşünülebilir.

- Elemanlar: birinci eleman $\{3\}$, ikinci eleman $\{4,5\}$, üçüncü eleman $\{8\}$. Boyut 3'tür.
- Öğeler: birinci elemanda 1, ikincide 2 (toplam 3), üçüncüde 1 öğe var (toplam 4). Uzunluk 4'tür.

4. Alt Dizi ve Üst Dizi

İki dizi alalım: $s_1 = \langle a_1 a_2 \dots a_r \rangle$ ve $s_2 = \langle b_1 b_2 \dots b_v \rangle$. s_1'in s_2'nin alt dizisi (subsequence) olması için şu koşul sağlanmalıdır: s_1'in her elemanı, s_2'nin **ayrı bir** elemanının alt kümesi olmalıdır.

Yani a_1, s_2'nin bir elemanının alt kümesi; a_2, s_2'nin sonraki ayrı bir elemanının alt kümesi; ve a_r'ye kadar böyle devam eder. Bu koşul sağlanıyorsa s_1, s_2'nin alt dizisidir. Ters yönden söylersek s_2, s_1'in üst dizisidir (supersequence).

Kitaptaki tanım bunu indislerle yazar: $1 \leq j_1 < j_2 < \cdots < j_r \leq v$ olacak şekilde indisler bulunmalı ve

$$a_1 \subseteq b_{j_1}, \quad a_2 \subseteq b_{j_2}, \quad \ldots, \quad a_r \subseteq b_{j_r}$$

olmalıdır.

Bu noktada bir öğrenci soru sordu.

Soru: s_1'deki a_1, a_2, a_3 elemanlarının s_2'nin elemanlarına sıralı bir şekilde mi alt küme olması gerekiyor? **Cevap:** Evet. a_1'in içindeki öğeler s_2'deki bir elemanın alt kümesi olmalı, bundan sonraki eleman da s_2'de daha sonraki bir elemanın alt kümesi olmalı. Elemanlar birbirine eşit de olabilir. Önemli olan kapsamadır: b elemanının içindeki öğeler a elemanını kapsamalıdır.

Soru: Buradan boyutların da eşit olması gerektiği sonucunu çıkarabilir miyiz? **Cevap (hocanın):** Tabii, çünkü hepsinin ayrı elemanların alt kümesi olduğunu söylüyoruz.

> **Ders sonrası not:** Hoca boyutların eşit olması gerektiğini söyledi; doğrusu boyutların eşit olması gerekmez. Tanımda $r \leq v$'dir. s_2 daha fazla elemana sahip olabilir. Yalnız s_1'in her elemanı, s_2'nin sırayı koruyan farklı bir elemanına eşlenmelidir. Örneğin boyutu 3 olan $\langle\{3\}\{4,5\}\{8\}\rangle$, boyutu 5 olan $\langle\{6\}\{3,7\}\{9\}\{4,5,8\}\{3,8\}\rangle$ dizisinin alt dizisidir: $\{3\} \subseteq \{3,7\}$, $\{4,5\} \subseteq \{4,5,8\}$, $\{8\} \subseteq \{3,8\}$ ve bu elemanlar aynı sırayla gelir (python ile denetlendi). "Ayrı eleman" koşulu boyut eşitliği değil, s_1'in iki elemanının s_2'nin aynı elemanına eşlenememesi anlamına gelir.

5. Problem Tanımı ve Müşteri Dizileri Örneği

Problem şöyledir: Girdi olarak veri dizileri ve kullanıcının verdiği bir en küçük destek (minimum support, minsup) verilir. Amaç, sıralı örüntüleri bulmaktır. Bir dizinin desteği, o diziyi alt dizi olarak içeren veri dizilerinin oranıdır.

Hoca bunu ilişkilendirme kurallarıyla karşılaştırdı. Orada en büyük sık öğe kümelerini bulmaya çalışıyorduk ve kuralları bunlardan çıkarıyorduk. Burada da **en büyük sık dizileri** bulmaya çalışırız.

5.1 İşlem Verisinden Müşteri Dizilerine

Hoca bir mağazadaki işlem verisini gösterdi. Müşteri 1'in iki farklı günde iki işlemi vardır: ilk gün bir öğe, ertesi gün bir öğe almıştır. Müşteri 2 ilk gün iki öğe, ikinci gün bir öğe, üçüncü gün üç öğe almıştır. Bu işlemler müşteriye ve zamana göre gruplanarak her müşteri için bir veri dizisine dönüştürülür:

Müşteri	Veri dizisi
1	$\langle\{30\}\{90\}\rangle$
2	$\langle\{10,20\}\{30\}\{40,60,70\}\rangle$
3	$\langle\{30,50,70\}\rangle$
4	$\langle\{30\}\{40,70\}\{90\}\rangle$
5	$\langle\{90\}\rangle$

Hoca bu gösterimin nasıl okunacağını vurguladı. Müşteri 2'nin dizisinde $\{40,60,70\}$ elemanı 30'dan **sonra** alınmıştır. Ama 40, 60 ve 70'in kendi aralarındaki sırasını bilemeyiz. Bildiğimiz tek şey, bu elemanın $\{30\}$ elemanından sonra geldiğidir.

5.2 Sık Diziler (minsup = %25)

minsup %25 olduğunda, 5 müşteri için bir dizinin en az $\lceil 0{,}25 \times 5 \rceil = 2$ müşteri dizisinde geçmesi gerekir (ara adım eklendi). Python ile tüm adaylar sayıldığında şu sık diziler çıkar:

Uzunluk	Sık dizi	İçeren müşteriler	Destek
1	$\langle\{30\}\rangle$	1, 2, 3, 4	%80
1	$\langle\{40\}\rangle$	2, 4	%40
1	$\langle\{70\}\rangle$	2, 3, 4	%60
1	$\langle\{90\}\rangle$	1, 4, 5	%60
2	$\langle\{30\}\{40\}\rangle$	2, 4	%40
2	$\langle\{30\}\{70\}\rangle$	2, 4	%40
2	$\langle\{30\}\{90\}\rangle$	1, 4	%40
2	$\langle\{40, 70\}\rangle$	2, 4	%40
3	$\langle\{30\}\{40, 70\}\rangle$	2, 4	%40

Hocanın asıl gösterdiği en büyük sık dizi, uzunluğu 3 olan $\langle\{30\}\{40, 70\}\rangle$'dir. Bu dizi müşteri 2'nin dizisinde ($\{30\} \subseteq \{30\}$, $\{40, 70\} \subseteq \{40, 60, 70\}$) ve müşteri 4'ün dizisinde ($\{30\} \subseteq \{30\}$, $\{40, 70\} \subseteq \{40, 70\}$) alt dizi olarak geçer. Destek $2/5 = \%40$, %25'in üzerindedir.

> **Ders sonrası not:** Hoca, dizinin sık sayılması için "beş işlemden ikiden fazlasında" geçmesi gerektiğini söyledi; doğrusu "en az ikisinde"dir. %25 eşiği $5 \times 0{,}25 = 1{,}25$ müşteriye karşılık gelir, yani 2 müşteri yeterlidir. $\langle\{30\}\{40, 70\}\rangle$ de tam 2 müşteride geçer.

Müşteri 3'ün dizisi $\langle\{30, 50, 70\}\rangle$ hem 30'u hem 70'i içerir, ama ikisi aynı elemandadır. Bu yüzden $\langle\{30\}\{70\}\rangle$'yi (30'dan sonra 70) desteklemez (ara adım eklendi).

6. GSP Algoritması ve Apriori ile Benzerliği

Hoca, iki algoritmanın benzerliğini görmek için önce Apriori algoritmasını yeniden gösterdi:

- Veri üzerinde ilk tarama (pass) yapılır, tek öğeli öğe kümelerinin sayıları bulunur.
- Buradan aday iki öğeli öğe kümeleri üretilir.
- İkinci tarama yapılır, iki öğeli öğe kümelerinin kaç kez geçtiği sayılır.
- Hangilerinin gerçekten sık olduğu denetlenir ve döngünün başına dönülür.

Ardından GSP algoritmasını gösterdi ve "aynı şey" dedi:

1. Bir tarama yapılır ve 1-diziler sayılır. Sık 1-diziler bulunur.
2. Döngüye girilir. GSP'de de bir aday üretme (candidate generation) fonksiyonu vardır. Sık 1-dizilerden aday 2-diziler üretilir.
3. Veri yeniden taranır, aday 2-dizilerin kaç kez geçtiği sayılır.
4. Hangilerinin gerçekten sık olduğu denetlenir ve döngünün başına dönülür. Böylece 1-dizilerden 2-dizilere, oradan 3-dizilere ilerlenir.
5. Algoritma bittiğinde bulunabilecek en büyük sık diziler elde edilir. Bunlardan sıralı kurallar (sequential rules) üretilir.

Örnekteki dizi için kural şöyle okunur: "30 gerçekleştiğinde, hemen ardından 40 ve 70 gerçekleşir."

Hocaya göre sıralı örüntü madenciliği algoritmalarının çoğu Apriori algoritmasının türevleridir ve ona benzer. Bu yüzden anlaşılmaları kolaydır. Bu algoritmaların birçoğu bugün kullanıcıların web sitelerindeki davranışını anlamak için kullanılmaktadır.

> **Ek bilgi (derste anlatılmadı):** GSP büyük olasılıkla "Generalized Sequential Patterns" adının kısaltmasıdır. Aday üretmede Apriori'deki aşağı kapanış (downward closure) özelliği dizilere uyarlanır. Bir dizi sıksa onun bütün alt dizileri de sıktır. Bu yüzden sık olmayan bir alt dizisi bulunan aday dizi sayılmadan budanır.

7. Tıklama Akışı Verisinde Sıralı Örüntüler

Hoca, bu algoritmaların nerede kullanıldığını göstermek için bir gezinme (browsing) verisi örneği gösterdi. A sayfası bir sitenin ana sayfası olsun. Bir kullanıcı gelir, önce ana sayfayı yeniler (refresh). Sonra bir bağlantıya tıklar ve B sayfasına geçer, onu da yeniler. Sonra başka bir bağlantıya tıklar ve aynı sitenin başka bir sayfası olan C'ye geçer, onu da yeniler. Kullanıcı sitede ilerledikçe gezinme verisi üretir.

Aynı veri bir tablo olarak da gösterildi. İlk sütun oturumu (session) gösterir. Birinci kullanıcı 0 numaralı oturumdadır:

- $t = 0$ anında A sayfasına gider.
- $3t$ anında A sayfasını yeniler.
- $4t$ anında B sayfasına geçer.
- Böyle devam ederek aynı oturumda farklı etkinlikler yapar.

Her oturum bir işlem (transaction) gibi düşünülür. Oturumun içindeki etkinlikler de o işlemin öğeleridir. Bütün oturumlara sıralı örüntü madenciliği uygulanınca, birçok kullanıcının oluşturduğu sık örüntüler bulunur. Slaytta bunlar arasında uzunluğu 1 olan, uzunluğu 2 olan ve uzunluğu 3 olan sık diziler vardı. Hoca, gerçek bir siteden toplanmış tıklama akışı verisi üzerinde sıralı örüntü madenciliğinin böyle kullanıldığını söyledi.

8. Gözetimli Öğrenme: Temel Kavramlar

Hoca ardından 3. bölüme, gözetimli öğrenmeye geçti ve bunun da web madenciliğinde kullanılacak önemli bir konu olduğunu söyledi.

Gözetimli öğrenmede etiketli (labeled) bir veriden öğrenilir ve yeni bir verinin etiketi, yani türü tahmin edilir. Hoca iki örnek verdi:

- **Acil servis:** Hastalar yüksek riskli ve düşük riskli olabilir. Yoğun bakım ünitesinde sınırlı sayıda yer vardır. Bu yüzden hangi hastanın kabul edileceğine dikkat etmek gerekir. Karar veren bir model gerekir: Hasta yüksek riskliyse kabul edilir, değilse yoğun bakıma alınmaz.
- **Kredi kartı başvurusu:** Her başvuruda kişinin yaşı, medeni durumu, yıllık geliri gibi veriler vardır. Başvurunun onaylanıp onaylanmayacağına dair ikili (binary) bir karar verilir.

Gözetimli öğrenme geçmiş deneyimlerden öğrenmeyi sağlar. Geçmiş deneyimlerden öğrenen bir model kurulur ve bu model bir hedef fonksiyonun (target function) sonucunu bulmak için kullanılır. Buradaki hedef fonksiyon tahmindir. Sonuç ikili olabilir: kredi başvurusunda onayla / onaylama, acil serviste yüksek risk / düşük risk.

8.1 Veri ve Süreç

Hoca kitaptaki örnek veriyi gösterdi. Veride farklı nitelikler (attribute) ve 15 veri öğesi vardır. Her veri öğesinin bir etiketi vardır. Hoca etiket (label) ile sınıfın (class) aynı anlama geldiğini söyledi. Burada iki sınıf vardır: Yes ve No. Bu veri modeli eğitmek için kullanılan eğitim verisidir (training data). Öğrendikten sonra bir model kurulur. Bu model, dört niteliğinin değerleri bilinen ama sınıfı bilinmeyen yeni bir veri öğesinin sınıfını tahmin etmek için kullanılır.

Etiketli veriden öğrenildiği için bu yönteme gözetimli öğrenme denir. "Gözetim" kısmı etiketli veriden gelir. Gözetimsiz öğrenmede ise etiket yoktur. Yine veriyi anlamaya çalışırız, bunun için de algoritmalar vardır.

Süreç şöyledir:

1. Eğitim verisine bir gözetimli öğrenme algoritması uygulanır ve bir model elde edilir.
2. Model, test verisi (test data) üzerinde kullanılır. Test verisi de eğitim verisi gibi etiketlidir, yani etiketleri biliriz.
3. Model her test öğesinin etiketini, etiketi bilinmiyormuş gibi tahmin eder.
4. Tahmin edilen etiketin gerçek etiketle aynı olup olmadığına bakılır ve doğruluk (accuracy) bulunur:

$$\text{Accuracy} = \frac{\text{doğru sınıflandırma sayısı}}{\text{toplam test örneği sayısı}}$$

Genellikle elimizdeki verinin bir kısmı eğitim, kalanı test verisi olarak ayrılır. Çoğu zaman verinin %70'i ya da %80'i eğitim, kalan %30'u ya da %20'si test verisi olarak kullanılır.

9. Öğrenme Ne Demektir?

Hoca öğrenmenin tanımını verdi. Bir veri kümesi D, bir görev T ve bir başarım ölçüsü M olsun. Bir bilgisayar sistemi, öğrendikten sonra T görevindeki başarımı M ile ölçüldüğünde artıyorsa, T görevini yapmak için D'den öğrenmiş sayılır. Yani öğrenme ancak hiç öğrenmemekten daha iyi sonuç veriyorsa başarılıdır.

Hoca bunu örnek veri üzerinde anlattı. Kaç etiket var? İki. Yeni verinin etiketini tahmin ederken iki şansımız var: Yes ya da No. Hiç öğrenme yapmadan %50 olasılıkla doğru tahmin edebiliriz. Bir modelin bu veriden öğrendiğini söylemek için tahmininin %50'den büyük olması gerekir. Aslında çok daha iyi olmalıdır, %80 ya da %90 gibi.

> **Hocanın vurgusu:** Öğrenilen model, sistemin görevi hiç öğrenme olmadan yaptığından daha iyi yapmasını sağlamalıdır. Hiç öğrenme yapmadan zaten %50 şansla tahmin edebiliyorsam, modelimin bundan daha iyi olmasını beklerim.

Kredi örneğinde:

- D: kredi başvuru verisi.
- T: kredinin onaylanıp onaylanmayacağını tahmin etmek.
- M: doğruluk.

Hoca ardından "öğrenmesiz" durumu daha dikkatli hesapladı. Verideki çoğunluk sınıfı Yes'tir. Hiçbir şey öğrenmeden her yeni başvuruya çoğunluk sınıfına bakarak "Yes" diyebiliriz. Bütün veri test verisi olarak kullanılırsa, 15 öğenin 9'u Yes olduğu için doğruluk

$$\frac{9}{15} = 0{,}60 = \%60$$

olur. Bu öğrenmesiz durumdur. Bir gözetimli öğrenme algoritması %60'tan daha iyi sonuç vermelidir.

> **Ders sonrası not:** Hoca önce iki sınıf olduğu için öğrenmesiz başarının %50 olduğunu söyledi, sonra çoğunluk sınıfına göre %60 hesapladı. Doğru taban çizgisi (baseline) çoğunluk sınıfının oranıdır, burada $9/15 = \%60$ (python ile denetlendi). %50, yalnız iki sınıf eşit sayıdaysa ya da rastgele tahmin ediliyorsa geçerlidir.

10. Eğitim ve Test Verisinin Dağılımı

Model eğitim verisinden öğrenip test verisinde sınanırken, eğitim örneklerindeki verinin dağılımının test verisindeki dağılımla aynı olması gerekir. Aksi hâlde modelin gerçek başarımı anlaşılamaz.

> **Hocanın vurgusu:** Test verisindeki öğelerin dağılımı eğitim verisindekine benzer olmalıdır. Bu çok önemlidir. İyi bir değerlendirme için test verisi eğitim verisini yeterince iyi temsil etmelidir.

Hoca, gözetimli öğrenmede birçok algoritma olduğunu, bunların bir kısmının bu derste, bir kısmının veri madenciliği dersinde, daha fazlasının da makine öğrenmesi dersinde öğrenileceğini söyledi. Burada amaç, öğrencilerin dönem projesinde topladıkları web verisine bu algoritmalardan birini uygulayabilmesiydi.

11. Karar Ağacı

İlk algoritma karar ağacıdır (decision tree). Hoca bunun etkili, uygulaması kolay ve yaygın kullanılan bir gözetimli öğrenme algoritması olduğunu söyledi. Kullanılan veri yine kredi verisiydi: 15 öğe, 9'u Yes, 6'sı No.

11.1 Model Bir Ağaç Veri Yapısıdır

Karar ağacı algoritmasının kurduğu model bir ağaç veri yapısıdır. Hoca, herkesin veri yapıları dersini aldığını, herhangi bir ağaç veri yapısının kullanılabileceğini söyledi. Önemli olan ağacın düğümlerine ne konacağı ve dallara ne gideceğidir.

Örnek ağaçta nitelikler düğümlere konmuştur:

- Kökte Age bulunur.
- Age = young dalında Has_job, Age = middle dalında Own_house, Age = old dalında Credit_rating bulunur.
- Toplam 4 karar düğümü vardır: bir kök ve üç dal düğümü.
- 7 yaprak düğüm (leaf node) vardır. Yapraklar sınıf etiketini alır, yani Yes ya da No olur.

11.2 Ağacın Kurulması

Başta bütün veri kök düğümdedir ve köke hangi niteliğin konacağına henüz karar verilmemiştir. Bir nitelik önem ölçütü kullanılır: bilgi kazancı (information gain), kazanç oranı (gain ratio) gibi. Bu ölçütler bir niteliğin veriyi dallarına ne kadar iyi dağıttığını ölçer. En önemli nitelik köke gelir.

Nitelikler düğümlere yerleştirildikten sonra 15 veri öğesi dağıtılır. Her nitelik bir soru gibidir: "Bu öğenin yaşı nedir?" Öğeler bu sorunun üç olası cevabına göre dağıtılır. Örneğin young değeri olan 5 öğe young dalına gider. Bu düğümde Has_job niteliği vardır ve true ya da false değerini alır. Bu 5 öğe Has_job değerlerine göre dağıtılır: 2'si bir yaprağa, 3'ü diğer yaprağa düşer. Bütün öğeler dallara dağıtıldıktan sonra her yaprağa düşen etiketlerin çoğunluğuna bakılır ve yaprağın etiketi bu olur.

Bu ağaç bir modeldir. Eğitim verisine karar ağacı algoritması uygulanmış ve bir ağaç veri yapısı olan model elde edilmiştir.

> **Ders sonrası not:** Hoca, nitelikleri bir kez bilgi kazancına göre sıraladığımızı, en önemlisini köke, sonrakileri (Has_job, Own_house, Credit_rating) alt düğümlere koyduğumuzu anlattı. Algoritma böyle çalışmaz. Bilgi kazancı her düğümde, o düğüme düşen veri alt kümesi için yeniden hesaplanır ve o alt küme için en iyi nitelik seçilir. Bu yüzden örnekte young dalında Has_job, middle dalında Own_house, old dalında Credit_rating çıkmıştır.

11.3 Modelle Tahmin

Yeni bir veri öğesi geldiğinde kökten başlanır ve öğenin değerlerine göre yaprağa kadar inilir. Hocanın örneğinde yeni öğenin Age değeri young olduğu için young düğümüne gidilir. Has_job değeri false olduğu için ilgili yaprağa ulaşılır. Bu yaprağın etiketi No'dur. Tahmin "No" olur.

11.4 Tek Bir Doğru Ağaç Yoktur

Aynı veriden başka bir ağaç da kurulabilir. Değişen şey, niteliklerin önemine karar veren ölçüttür. Bilgi kazancı yerine başka bir ölçüt kullanılırsa köke gelecek nitelik değişebilir. Yani karar ağacı algoritması, kullandığı nitelik önem ölçütüne göre parametrik bir algoritmadır.

> **Hocanın vurgusu:** Tek bir doğru karar ağacı vardır diyemeyiz. Birden fazla ağaç kurulabilir ve her birinin tahmin doğruluğu farklı olur.

Hoca örnek olarak iki karar düğümü ve üç yaprağı olan farklı bir ağaç gösterdi.

11.5 Ağaçtan Kural Çıkarma

Karar ağacı bellekte bir ağaç veri yapısı olarak tutulabilir. Ya da ağaçtan, önceki dersteki sınıf ilişkilendirme kuralları gibi kurallar çıkarılıp model olarak kullanılabilir. Bu kurallar sıralı yazılır: önce kökten çıkan kurallar, sonra onun dallarındaki karar düğümlerinden çıkan kurallar. Yeni gelen verinin etiketi bu kural zinciriyle tahmin edilir.

11.6 Böl ve Yönet

Karar ağacı öğrenme algoritması bir böl ve yönet (divide and conquer) algoritmasıdır. Hoca böl ve yönet algoritmalarının algoritma tasarımı dersinde ayrıntılı görüleceğini söyledi. Bu tür algoritmalar veriyi her aşamada böler ve her parçaya aynı işlemi, bir çözüme ulaşana kadar tekrar tekrar uygular. Karar ağacında da veri her aşamada bölünür: önce yaşın young, middle ya da old olmasına göre, sonra aynı işlem karar düğümlerinde tekrarlanır.

> **Ders sonrası not:** Hoca böl ve yönetin veriyi her aşamada ikiye böldüğünü söyledi. Karar ağacında veri, seçilen niteliğin değer sayısı kadar parçaya bölünür. Age için bu üç parçadır (young, middle, old).

Nitelik önem ölçütüne saflık bozukluğu fonksiyonu (impurity function) da denir. Algoritmanın durma koşulları şunlardır:

- Düğüme düşen bütün örnekler aynı sınıftandır.
- Kullanılacak nitelik kalmamıştır.
- Örnek kalmamıştır.

Hoca, örnek ağaçta her daldaki bütün veri öğelerinin aynı sınıfa ait olduğunu, yani birinci durma koşulunun sağlandığını gösterdi. Ardından algoritmanın sözde kodunu (C4.5) gösterdi ve önceki slayttakileri tekrarladığını söyledi.

> **Ek bilgi (derste anlatılmadı):** Bilgi kazancı, bir düğümdeki sınıf dağılımının entropisinden, nitelikle bölündükten sonraki dalların ağırlıklı ortalama entropisinin çıkarılmasıyla bulunur: $\text{gain}(D, A) = \text{entropy}(D) - \sum_j \frac{|D_j|}{|D|} \text{entropy}(D_j)$, burada $\text{entropy}(D) = -\sum_c \Pr(c) \log_2 \Pr(c)$. Kazanç oranı, çok değerli niteliklerin kayırılmasını önlemek için bu kazancı bölünmenin kendi entropisine böler.

11.7 Nitelik Seçiminin Sezgisi: Yanlış Yerleşen Örnekler

Hoca bir karar düğümünde hangi niteliğin seçileceğini sezgisel olarak gösterdi. Ölçü, çoğunluk etiketinden farklı olan, yani "yanlış yerleşen" (misplaced) örnek sayısıdır.

Age seçilirse:

- Veri üç dala dağılır, her dala 5 öğe düşer.
- Çoğunluk etiketleri: young dalında No, middle dalında Yes, old dalında Yes.
- Yanlış yerleşen örnekler: young dalında 2 (2 Yes, 3 No), middle dalında 2 (3 Yes, 2 No), old dalında 1 (4 Yes, 1 No).
- Toplam: $2 + 2 + 1 = 5$.

Own_house seçilirse:

- true dalına 6 öğe, false dalına 9 öğe düşer.
- true dalının çoğunluk etiketi Yes'tir ve 6 öğenin hepsi Yes'tir. Yanlış yerleşen 0.
- false dalının çoğunluk etiketi No'dur. 9 öğenin 6'sı No, 3'ü Yes'tir. Yanlış yerleşen 3.
- Toplam: $0 + 3 = 3$.

Own_house ile 3, Age ile 5 öğe yanlış yerleşir. Bu yüzden bu düğüm için Own_house daha iyi bir seçimdir. Hoca, bir karar düğümünde hangi niteliğin seçileceğine böyle karar verildiğini söyledi. Değerler kitaptaki kredi verisi üzerinde python ile denetlendi.

12. Sınıf İlişkilendirme Kurallarıyla Sınıflandırma

İkinci yöntem, sınıf ilişkilendirme kurallarıyla sınıflandırmadır (classification based on class association rules). Hoca bunun karar ağacına benzediğini söyledi. Önceki bölümdeki sınıf iliş-

kilendirme kuralı madenciliği algoritmasıyla, verilen en küçük destek ve en küçük güven (minimum confidence) değerleri için sağ tarafı bir sınıf etiketi olan kurallar çıkarılır. Bu kural kümesi model olarak kullanılır.

Yöntem şöyle çalışır:

1. Çıkarılan sınıf ilişkilendirme kuralları güven ve destek değerlerine göre büyükten küçüğe sıralanır.
2. Yeni bir veri öğesi geldiğinde ilk kurala bakılır: Bu kural yeni öğe hakkında bir yanıt veriyor mu, yani öğeyi kapsıyor mu?
3. Vermiyorsa bir sonraki kurala, o da vermiyorsa bir sonrakine geçilir. Öğeyi ilk kapsayan kuralın sınıfı atanır.
4. Hiçbir kural öğeyi yanıtlayamıyorsa varsayılan sınıf (default class) atanır. Yes ya da No bir yanıt vermek zorunda olduğumuz için bu gereklidir.

Önceki modelde içinde karar ağacı olan bir model vardı. Bu yöntemde ise model kurallardan oluşur. Hoca önceki derste anlatılan sınıf ilişkilendirme kuralı madenciliği algoritmasının sözde kodunu da yeniden gösterdi.

13. Naive Bayes Sınıflandırıcı

Üçüncü gözetimli öğrenme algoritması model kurmak için istatistik kullanır. Karar ağacı modeli bir ağaç veri yapısıydı, sınıf ilişkilendirme kuralları modeli bir kural kümesiydi. Burada istatistik denklemleriyle bir model kurulur.

13.1 Veri ve Amaç

Hoca yeni bir örnek veri kümesi gösterdi. Dört nitelik vardır ve etiket, kişinin bilgisayar alıp almadığıdır: buys_computer = yes ya da buys_computer = no. Toplam 14 veri öğesi vardır. Etiketi belirlenecek yeni örnek şudur:

$$X = (\text{age} \leq 30,\ \text{income} = \text{medium},\ \text{student} = \text{yes},\ \text{credit_rating} = \text{fair})$$

İki sınıf vardır: C_1 = buys_computer = yes, C_2 = buys_computer = no. Asıl hesaplamak istediğimiz, X verildiğinde sınıfın ne olduğudur: $P(C_1 \mid X)$ ve $P(C_2 \mid X)$.

13.2 Bayes Teoremi ve Sadeleştirme

Bunun için Bayes teoremi (Bayes' theorem) kullanılır:

$$P(C_i \mid X) = \frac{P(X \mid C_i)\, P(C_i)}{P(X)}$$

Bu denklem C_1 ve C_2 için ayrı ayrı yazılır. Hangisi büyükse X o sınıfa atanır. $P(C_1 \mid X) > P(C_2 \mid X)$ ise buys_computer = yes seçilir.

İki denkleme bakınca ortak bir terim görülür: Paydadaki $P(X)$ ikisinde de aynıdır. Yalnız karşılaştırma yapacağımız için $P(X)$ atılabilir. Böylece yalnız payları hesaplayıp karşılaştırmak yeter:

$$P(X \mid C_i)\, P(C_i)$$

Bu ifadenin iki parçası vardır.

Sınıf olasılığı $P(C_i)$: Bu çok kolaydır. Toplam 14 öğenin 9'u buys_computer = yes, kalan 5'i no'dur:

$$P(C_1) = \frac{9}{14} \approx 0{,}643, \qquad P(C_2) = \frac{5}{14} \approx 0{,}357$$

Sınıf verildiğinde X'in olasılığı $P(X \mid C_i)$: Hoca X'in her niteliğini ayrı bir öğe gibi ele aldığımızı söyledi. Sınıf verildiğinde her niteliğin görülme olasılığı ayrı ayrı hesaplanır ve hepsi çarpılır:

$$P(X \mid C_i) = \prod_j P(x_j \mid C_i)$$

> **Ek bilgi (derste anlatılmadı):** Olasılıkları böyle çarpmak, niteliklerin sınıf verildiğinde birbirinden koşullu bağımsız olduğunu varsaymak demektir. Yönteme "naive" (saf) denmesinin sebebi bu varsayımdır. Gerçek veride nitelikler çoğu zaman bağımsız değildir, ama yöntem pratikte yine de iyi çalışır.

Hoca bir niteliğin olasılığını tablo üzerinde saydı. 9 tane buys_computer = yes öğesi vardır. Bunların kaçında birinci nitelik age ≤ 30'dur? Tabloda sayınca 2 öğe bulunur. Demek ki

$$P(\text{age} \leq 30 \mid \text{yes}) = \frac{2}{9}$$

Diğer niteliklerin olasılıkları da aynı şekilde hesaplanır.

13.3 Hesabın Tamamı

Hoca hesabın sonucunu slaytta gösterdi ve "yes" sınıfının olasılığının daha büyük çıktığını söyledi. Ara değerler şöyledir:

Nitelik değeri	$P(\cdot \mid \text{yes})$	$P(\cdot \mid \text{no})$
age ≤ 30	$2/9 \approx 0{,}222$	$3/5 = 0{,}6$
income = medium	$4/9 \approx 0{,}444$	$2/5 = 0{,}4$
student = yes	$6/9 \approx 0{,}667$	$1/5 = 0{,}2$
credit_rating = fair	$6/9 \approx 0{,}667$	$2/5 = 0{,}4$

Çarpımlar:

$$P(X \mid \text{yes}) = 0{,}222 \times 0{,}444 \times 0{,}667 \times 0{,}667 \approx 0{,}044$$

$$P(X \mid \text{no}) = 0{,}6 \times 0{,}4 \times 0{,}2 \times 0{,}4 = 0{,}0192 \approx 0{,}019$$

Sınıf olasılıklarıyla çarpınca:

$$P(X \mid \text{yes})\, P(\text{yes}) \approx 0{,}044 \times 0{,}643 \approx 0{,}028$$

$$P(X \mid \text{no})\, P(\text{no}) \approx 0{,}019 \times 0{,}357 \approx 0{,}007$$

$0{,}028 > 0{,}007$ olduğu için X, buys_computer = yes sınıfına atanır. Değerler python ile kesirli sayılarla hesaplanıp doğrulandı ($0{,}0439$, $0{,}0192$, $0{,}0282$, $0{,}0069$).

Hocaya göre bu, öğrenmesi ve uygulaması kolay bir algoritmadır. Gereken iki olasılık grubu, $P(C_i)$ ve $P(x_j \mid C_i)$, doğrudan verilen veriden hesaplanır. Bunlar hesaplanınca yeni bir öğenin etiketi hemen bulunur. Bu yönteme Naive Bayes sınıflandırıcı (Naive Bayesian classifier) denir.

13.4 Üç Model Türü

Hoca şimdiye kadarki üç yöntemi model türüne göre karşılaştırdı:

Yöntem	Model
Karar ağacı	Bir veri yapısı: karar ağacı
Sınıf ilişkilendirme kuralları	Kural kümesi
Naive Bayes	Veriden hesaplanan olasılıklar kümesi

14. k En Yakın Komşu (kNN)

Son gözetimli yöntem k en yakın komşudur (k-nearest neighbor, kNN). Hoca bunun ilk üç yöntemden farklı olduğunu vurguladı.

> **Hocanın vurgusu:** İlk üç yöntem bir model kurar, kNN kurmaz. kNN veriyi model olarak kullanır. Her yeni öğe geldiğinde algoritma veriye döner ve yeni öğenin etiketini tahmin etmek için veriden bazı öğeler bulur.

Yöntem şöyle çalışır. Başta kullanıcı bir k değeri verir, örneğin $k = 6$. Adı da buradan gelir. Yeni bir veri öğesi geldiğinde, bir uzaklık fonksiyonu (distance function) kullanılarak ona en yakın 6 veri noktası bulunur. Bu 6 noktanın sınıf etiketlerinin çoğunluğuna bakılır. Hocanın örneğinde çoğunluk yeşil sınıftı. Yeni öğenin etiketi çoğunluk sınıfı olarak tahmin edilir.

Karar ağacı, sınıflandırma kuralları ve Naive Bayes eğitim verisine uygulandığında bir model kurar ve bu model test verisinde kullanılır. kNN'de ise eğitim ve tahmin iç içedir. Model kurulmaz. Her yeni veri için eğitim verisine dönülür.

Sözde kod:

- Girdi: veri kümesi D, etiketi tahmin edilecek öğe d ve k değeri.
- d ile D'deki her öğe arasındaki uzaklık hesaplanır.
- D'de d'ye en yakın k örnek seçilir.
- Bu k örnekteki çoğunluk sınıfı d'ye atanır.

> **Ders sonrası not:** Hoca kNN için "üçüncü tür sınıflandırıcı" dedi; sıralamada dördüncü yöntemdir (karar ağacı, sınıf ilişkilendirme kuralları, Naive Bayes, kNN).

15. Gözetimsiz Öğrenme ve Kümeleme

Aradan sonra hoca gözetimsiz öğrenmeye geçti. Gözetimli öğrenmede veriden öğreniyorduk, veri bir anlamda öğretmendi, çünkü verinin etiketi vardı. Gözetimsiz öğrenmede verinin etiketi, yani hedef niteliği (target attribute) yoktur. Hoca hedef nitelik ile etiketin aynı şey olduğunu söyledi.

Kümelemede (clustering) amaç, birlikte gruplanabilecek veri öğelerini keşfetmektir. Bir veri grubuna küme (cluster) denir. Hocanın oyuncak örneğinde veri öğeleri iki boyutluydu ve iki doğal küme gözle görülüyordu. İki ya da üç boyutta veri görselleştirilip belirgin gruplar olup olmadığı görülebilir. Gerçek veride ise her veri noktasının çok sayıda boyutu vardır. Bu yüzden kümeleri görmek zordur ve kümeleme algoritmalarına güvenmek gerekir.

15.1 Kümeleme Neden Kullanılır?

Kümeleme, verideki yapıyı anlamak ve buna göre eylemler yapmak için kullanılır. Hocanın örnekleri:

- **Üretim:** Tişört üretiyorsanız, insanların hangi bedenleri giymek istediğini bilmek istersiniz. Buna göre hangi bedenleri üreteceğinize karar verirsiniz.
- **Pazarlama:** Müşterilerin hangi segmentlere ayrıldığını bulup pazarlamayı bu segmentlere göre yapmak.
- **Belgeler:** Word, metin ya da HTML belgelerinden oluşan bir küme içeriklerine göre kategorilere ayrılabilir.
- **Web madenciliği:** Kümeleme web madenciliğinde çok önemli bir konudur. Bir sitedeki kullanıcı davranışını ve sitenin sıcak noktalarını (hotspots), yani kullanıcıların en çok kullandığı işlevleri ve sayfaları anlamak için kullanılır.

15.2 Kümeleme Algoritmalarının Kategorileri

Hoca kümeleme algoritmalarının farklı kategorilerini saydı:

- Bölümlemeye dayalı (partition-based / partitional) kümeleme.
- Hiyerarşik (hierarchical) kümeleme.
- Dağılıma dayalı (distribution-based) kümeleme.

Başka türler de vardır, ama dersin odağı bu kategorilerin temsilci algoritmalarıdır.

> **Ders sonrası not:** Hoca hiyerarşik kümelemeyi gösterirken "bu dağılıma dayalı kümeleme" dedi ve ileride hiyerarşik kümelemeyi "verinin dağılımına dayalı" diye andı. Bunlar ayrı kategorilerdir. Hiyerarşik kümeleme bir ağaç (dendrogram) üretir. Dağılıma dayalı kümeleme ise (örneğin Gauss karışım modelleri) veriyi olasılık dağılımlarıyla açıklar.

16. Kümeleme Kalitesi

Bir kümeleme algoritması veriye uygulandığında örneğin üç küme bulunur. Dört, beş ya da daha fazla küme de bulunabilirdi. Bu yüzden kümeleme sonucunun kalitesini ölçmek gerekir: Bulunan kümeler en iyi kümeler mi? Bunun için iki ölçü kullanılır:

- **Kümeler arası uzaklık (inter-cluster distance):** Ayrı kümeler arasındaki uzaklıktır.
- **Küme içi uzaklık (intra-cluster distance):** Bir kümenin elemanları arasındaki ortalama uzaklıktır.

Küme içi uzaklığın olabildiğince küçük olması istenir. Kümeler olabildiğince yoğun olmalı, kümedeki bütün noktalar birbirine olabildiğince yakın olmalıdır. Kümeler arası uzaklığın ise olabildiğince büyük olması istenir, yani kümeler birbirinden ayrık olmalıdır.

> **Ders sonrası not:** 01:54:32 civarında hoca bir an tersini söyledi ("inter-cluster minimize, intra-cluster maximize"). Doğrusu: küme içi uzaklık en küçük, kümeler arası uzaklık en büyük olmalıdır. Hoca bunu hemen öncesinde doğru anlatmıştı.

Kümelemenin kalitesi kullanılan algoritmaya ve veriye bağlıdır. Her veri kümesinde iyi kümeleme yapan tek bir mükemmel algoritma yoktur. Farklı türde veri kümelerinde farklı kümeleme algoritmaları kullanılır. Kalite uzaklık fonksiyonuna da bağlıdır, çünkü kümeleme algoritmalarında uzaklık fonksiyonu çok önemli bir etkendir.

17. k-Means Algoritması

Bölümlemeli algoritmaların temsilcisi olarak k-means anlatıldı. Hoca bunun başka derslerde de duyulmuş olabileceğini, burada bir tekrar olduğunu söyledi.

17.1 Girdi

Veri kümesi $D = \{x_1, x_2, \dots, x_n\}$'dir, yani n veri örneği vardır. Her örnek r boyutlu bir vektördür:

$$x_i = (x_{i1}, x_{i2}, \dots, x_{ir})$$

Oyuncak örnekte $r = 2$ idi. Gerçek veride, özellikle web madenciliğinde boyut sayısı çok yüksektir.

k-means, kullanıcının bulunacak küme sayısını baştan girmesini bekler. Bu parametre k'dir. Algoritma veri noktalarını k merkez (centroid) etrafında kümeler.

17.2 Algoritma

k-means yinelemeli (iterative) bir algoritmadır:

```
Algorithm k-means(k, D)
  choose k data points as the initial centroids (cluster centers)
  repeat
      for each data point x in D do
          compute the distance from x to each centroid
          assign x to the closest centroid
      re-compute the centroids using the current cluster memberships
  until the stopping criterion is met
```

Adımlar:

1. Başta k farklı veri noktası başlangıç merkezleri olarak seçilir. Bunlara tohum (seed) denir.
2. Döngüye girilir. Bütün veri kümesi taranır ve her veri öğesi en yakın merkeze atanır.
3. Atamadan sonra her kümenin merkezi yeniden hesaplanır.
4. Yakınsama (convergence) ölçütü sağlanmamışsa döngünün başına dönülür.

17.3 Oyuncak Örnek: k = 2

Hoca algoritmanın iki boyutlu bir oyuncak örnekte nasıl çalıştığını adım adım gösterdi. $k = 2$ verildiği için sonunda iki küme olacaktır.

- **Başlangıç:** Algoritma rastgele iki veri noktasını başlangıç merkezi seçer.
- **1. tur, atama:** Veri tabanındaki bütün veri öğeleri tek tek en yakın merkeze atanır. Dört veri noktası birinci merkeze daha yakın olduğu için ona, kalan noktalar ikinci merkeze atanır.
- **1. tur, merkez hesabı:** Yeni atamalara göre merkezler yeniden hesaplanır. Yeni merkez, kümedeki bütün noktaların ortalamasıdır: x ekseni için ortalama ve y ekseni için ortalama. Bulunan merkez gerçek bir veri noktası değil, "sanal" (virtual) bir noktadır. Başta gerçek veri noktaları merkez seçilmişti, birinci turdan sonra sanal merkezler bulunur.
- **2. tur:** Veri tabanı yeniden taranır, her nokta en yakın merkeze atanır ve merkezler yeniden hesaplanır.
- **3. tur:** Üçüncü tarama yapılır, noktalar atanır ve merkezler yeniden hesaplanır. Bu turda sanal merkezlerin yeri değişmez. Bu, k-means'in durma ölçütlerinden biridir.

Hoca ardından sözde koda döndü. Dış döngünün içinde bir iç döngü vardır. İç döngü veri kümesindeki bütün öğeleri dolaşır, her x öğesi ile her merkez arasındaki uzaklığı hesaplar ve öğeyi en yakın merkeze atar. İç döngü bittikten sonra kümelerin merkezleri küme üyeliklerine göre yeniden hesaplanır ve döngünün başına dönülür.

18. Durma Ölçütleri ve Uzaklık Fonksiyonu

18.1 Üç Durma Ölçütü

Her turdan sonra, döngünün başına dönmeden önce algoritmanın bitip bitmediği denetlenir. Bunun için durma ya da yakınsama ölçütü (stopping / convergence criterion) kullanılır. Üç ölçüt vardır:

1. **Veri noktalarının farklı kümelere yeniden atanmaması ya da çok az atanması.** Örnekte üçüncü turda yeni merkezlere yeniden atama yapıldığında hiçbir veri noktası küme değiştirmez. Aynı noktalar önceki kümelerinde kalır. Bu ölçüt kullanılıyorsa burada durulabilir.
2. **Merkezlerin değişmemesi ya da çok az değişmesi.** Örnekte atamadan sonra merkezler yeniden hesaplandığında değerleri değişmez. Bu ölçüt kullanılıyorsa burada durulur.
3. **Hata kareleri toplamındaki (sum of squared error, SSE) azalmanın çok küçük olması.** SSE şöyle tanımlanır:

$$\text{SSE} = \sum_{j=1}^{k} \sum_{x \in C_j} \text{dist}(x, m_j)^2$$

Burada C_j, j. küme, m_j de onun merkezidir. Hoca hesabı şöyle anlattı: Her turun sonunda bir kümedeki her nokta ile kümenin merkezi arasındaki uzaklık hesaplanır ve karesi alınır. Bu,

birinci küme için yapılır, ikinci küme için tekrarlanır ve hepsi toplanır. Bu bir değer verir. İkinci turun sonunda aynı hesap başka bir SSE değeri verir. İki SSE arasındaki değişim çok küçükse yineleme durdurulur.

Algoritma gerçekleştirilirken bu ölçütlerden biri kullanılır. Sözde koddaki dış döngünün "until" kısmı bu durma ölçütüdür.

18.2 Uzaklık Fonksiyonu

Merkez ile kümedeki her veri noktası arasındaki uzaklığı hesaplamak için bir uzaklık fonksiyonu seçmek gerekir. Bu yüzden uzaklık fonksiyonu çok önemlidir. En yaygın kullanılanı Öklid uzaklığıdır (Euclidean distance). Bu fonksiyon bütün nitelik değerlerinin sayısal olduğunu varsayar. Ortalama da buna göre hesaplanabilir. Hoca formülün bilindiğini söyleyip tekrar etmedi.

19. Disk Üzerindeki Veride k-Means ve Karmaşıklık

19.1 Veriyi Belleğe Almadan k-Means

k-means, veri kümesindeki çok sayıda noktaya ölçeklenebilen bir algoritmadır. Bütün veriyi belleğe almak gerekmez. Veri fiziksel depolamada durabilir ve k-means yine de çalıştırılabilir. Hoca bunun nasıl yapıldığını anlattı:

1. Başta k küme için birer başlangıç merkezi seçilir. Bunlara m_j denir.
2. Dış döngüye girilir. Her küme için iki değer sıfırlanır: bir toplam (sum) ve bir sayaç (count).
3. İç döngüde veri kümesi üzerinde bir tarama yapılır. Her veri öğesi x ile k merkezin her biri arasındaki uzaklık hesaplanır ve k farklı değer çıkar. En küçük uzaklık seçilir, x o kümeye atanır ve o kümenin toplam değeri ile sayacı güncellenir. Aynı işlem ikinci, üçüncü, dördüncü öğe için tekrarlanır.
4. Veri kümesinin tamamı bitince her küme için yeni merkez hesaplanır:

$$m_j = \frac{\text{sum}_j}{\text{count}_j}$$

5. Durma ölçütü denetlenir. Sağlanmamışsa döngünün başına dönülür.

Böylece veri belleğe sığmayacak kadar büyük olsa bile veri tabanında durabilir ve k-means çalıştırılabilir. Toplam/sayaç güncellemesiyle bulunan merkezin, kümedeki noktaların doğrudan ortalamasıyla aynı olduğu python (numpy) ile küçük bir sınama verisinde denetlendi.

19.2 Zaman Karmaşıklığı

k-means yinelemelidir ve durma ölçütü sağlanana kadar kaç tur gerektiğini önceden bilemeyiz. Karmaşıklığın parametreleri şunlardır:

- t: tur (yineleme) sayısı.
- k: küme sayısı. Her taramada her x için k merkezle karşılaştırma, yani k işlem yapılır.
- n: veri noktası sayısı. Bu karşılaştırmalar her taramada n kez yapılır.

Zaman karmaşıklığı:

$$O(t\,k\,n)$$

k ve t'nin küçük olması beklenir. Bu yüzden k-means aslında doğrusal (linear) bir algoritmadır, yani zaman karmaşıklığı veri sayısına bağlıdır. Etkili, anlaşılması ve gerçekleştirilmesi kolay bir algoritmadır.

20. k-Means'in Zayıf Yönleri ve Çözümleri

Hoca k-means'in zayıf yönlerini saydı:

- k **önceden bilinmeli.** Olası kümeler hakkında önceden bilgi gerekir. Ama veri analiz edilmeden kaç küme bulunacağını bilmek zordur. Bu yüzden k-means'ten önce en uygun k'yi bulan başka algoritmalar çalıştırılır.
- **Aykırı değerler (outliers).** Veride bir aykırı değer varsa k-means'in sonucunu etkiler.
- **Yalnız ortalaması hesaplanabilen veriye uygulanabilir.** Verinin nitelikleri sayısal olmalıdır.

20.1 Aykırı Değer Sorunu

Hoca iki doğal kümesi olan bir veri kümesi gösterdi. Veride bir aykırı değer olduğu için k-means doğru olmayan iki küme bulur. İdeal kümeleme, iki doğal küme ve hiçbir kümeye uymayan bir aykırı noktadır.

Bu sorun için iki yöntem vardır:

1. **Aykırı değerleri bulup çıkarmak:** k-means'ten önce aykırı değer bulma ve çıkarma (outlier detection and removal) algoritmaları uygulanır.
2. **Rastgele örneklem (random sampling):** Önce rastgele bir örneklem alınır, kümeleme bu örneklem üzerinde yapılır, sonra kalan noktalar bu kümelere atanır.

Hoca ikinci yöntemi bir örnekle anlattı. Elimizde içinde aykırı değerler olan 1 milyon veri noktası olsun. Bunların %10'u rastgele seçilip belleğe getirilir ve önce bunlar üzerinde kümeler bulunur. Diyelim ki 5 küme bulundu ve bu 5 kümenin merkezleri biliniyor. Kalan veri bu 5 kümeye dağıtılır. Kalan veri için kümeleme algoritması yeniden çalıştırılmaz, yalnız atama yapılır. Fikir şudur: 1 milyon veride aykırı değer olabilir, ama rastgele seçilen %10'un içinde bulunma olasılığı düşüktür.

20.2 En Uygun k

k seçme sorunu için en uygun k'yi bulan yöntemler vardır: elbow ve silhouette. Hoca ayrıntısına girmedi. Anlattığına göre bu yöntemler k'yi 2'den başlatıp 2, 3, 4, 5, 6, 7, 8, 9 diye artırır ve en yüksek kaliteli kümelemeyi veren k değerini bulmaya çalışır. En uygun k bulunduktan sonra k-means çalıştırılır.

> **Ders sonrası not:** Hoca elbow ve silhouette'i "algoritma" diye andı ve "en yüksek kaliteli kümelemeyi veren k'nin" seçildiğini söyledi. Bunlar k seçme yöntemleridir. Elbow yönteminde k arttıkça SSE hep azalır. Bu yüzden en düşük SSE değil, SSE'deki azalmanın belirgin biçimde yavaşladığı "dirsek" noktası seçilir. Silhouette yönteminde ortalama silhouette değeri en yüksek olan k seçilir.

20.3 Başlangıç Tohumlarının Seçimi

En uygun k bilinse bile, doğru kümeleme sonucuna ulaşmak için başlangıç merkezlerinin doğru seçilmesi gerekir. Hoca iki belirgin kümesi olan iki boyutlu bir örnek gösterdi. Başta seçilen iki veri noktasıyla k-means çalıştırıldığında algoritma beklenmeyen iki küme bulur. Farklı başlangıç tohumları seçilseydi k-means doğal kümeleri bulabilecekti. Yani başlangıç tohumlarının seçimi çok önemlidir.

Hocanın anlattığı tohum seçme yöntemi şöyledir:

1. Veri kümesinden bir veri öğesi birinci tohum olarak seçilir.
2. Veri taranır. Her öğenin birinci tohuma uzaklığı bulunur. Birinciden en uzak öğe ikinci tohum olur.
3. $k = 3$ ise üçüncü tohum için veri yeniden taranır. Bu kez birinci ve ikinci tohumun ikisinden de ortalamada en uzak öğe seçilir.
4. Böyle devam edilir. Birbirinden en uzak öğeler tohum seçildikten sonra k-means çalıştırılır.

> **Ek bilgi (derste anlatılmadı):** Bu "en uzak nokta" (farthest-first) yöntemi aykırı değerlere duyarlıdır, çünkü en uzak nokta çoğu zaman bir aykırı değerdir. Bu yüzden

önce aykırı değerleri temizlemek ya da örneklem kullanmak önemlidir. Bugün yaygın kullanılan k-means++ yönteminde tohumlar, en yakın tohuma uzaklığın karesiyle orantılı olasılıkla rastgele seçilir.

20.4 k-Means'i İyi Kullanmak İçin

Hoca k-means'i en iyi şekilde kullanmak için yapılması gerekenleri toparladı:

Hocanın vurgusu:

- Veri bir kez taranır, aykırı değerler varsa çıkarılır. Aykırı değer çıkarmayla uğraşmak istenmiyorsa rastgele örneklem alınır. Kümeleme örneklem üzerinde yapılır, kalan öğeler yeniden kümelemeye sokulmadan örneklemde oluşan kümelere atanır.
- En uygun k, elbow ya da silhouette gibi bir yöntemle bulunur.
- Başlangıç tohumları birbirinden olabildiğince uzak seçilir.

21. Kümelerin Şekli ve Algoritma Seçimi

Bütün bunlar yapılsa bile k-means her şekli bulamaz. Hoca iki belirgin kümesi olan bir örnek gösterdi. Zayıf yönlerin hepsi giderildikten sonra bile k-means istenmeyen iki küme bulur. k-means ve genel olarak bölümlemeli kümeleme algoritmaları yalnız eliptik ya da yuvarlak şekilli kümeleri bulabilir. Farklı şekilli kümeler için hiyerarşik kümeleme gibi başka algoritmalar gerekir.

Bu noktada sohbet kanalından bir soru geldi.

Soru: Kümeleme yaparken elde edeceğimiz kümelerin eliptik şekilli olup olmadığını nasıl anlayacağız? **Cevap:** Bunu önceden bilemeyiz. Kümelemeyi uygulamadan önce sonuçta eliptik mi eliptik olmayan mı kümeler çıkacağını bilmeyiz. Bu yüzden farklı kategorilerden temsilci algoritmalar uygulamak gerekir: k-means, bisecting k-means gibi hiyerarşik bir algoritma, agglomerative kümeleme gibi başka bir hiyerarşik algoritma. Sonuçları görmeden hangi algoritmanın en iyi kümelemeyi bulacağını bilemeyiz. Bunu yaparken farklı uzaklık ölçüleri de denenmeli ve daha iyi bir kümeleme sonucu çıkıp çıkmadığına bakılmalıdır.

Hocanın vurgusu: Kümelemede en iyi yol, farklı kategorilerden temsilci algoritmalar seçip veri kümesine uygulamak ve farklı uzaklık ölçüleri denemektir. Projesinde web verisi üzerinde kümeleme yapan gruplardan da farklı kümeleme algoritmaları ve uzaklık ölçüleriyle deney yapmaları ve sonuçları farklı ölçütlerle değerlendirmeleri beklenir.

22. Küme Gösterimi

Kümeler bulunduktan sonra gösterilmeleri gerekir. Bütün veri öğeleri bellekte tutulmaz. Veri kümesinde örneğin iki küme olduğunu biliriz, ama kümeyi kullanıcıya gösterebilmek için onu tanımlayacak bir yol gerekir. Hocanın saydığı yollar:

- **Merkez ve yarıçap:** Her kümenin merkez noktası ve yarıçapı kümeyi temsil eder.
- **Her boyut için sınır değerleri:** Veri noktalarının her boyutu için sınır noktaları kullanılır. Hocanın iki boyutlu örneğinde her noktanın x ve y değeri vardı ve kümeleme sonucunda üç küme bulunmuştu. Birinci küme "bütün noktaların x değeri 2'den küçük" diye, ikinci küme "x değeri 2'den büyük ve y değeri 1,5'ten büyük" diye gösterilir.
- **En sık geçen öğe:** Bazen kümede en sık görülen öğe kümenin temsilcisi olarak kullanılır. Metin kümelemede, örneğin 100 belgelik bir kümede 3 küme bulunduysa, her küme içinde en sık geçen sözcüklerle temsil edilir.

Hoca, bir kümenin bütün öğelerinin bellekte tutulmadığını, yalnız veri kümesindeki kümeleri temsil eden bilgilerin kullanıldığını tekrar hatırlattı.

23. Sonraki Dersler ve Proje Fikirleri

Eliptik olmayan şekilleri bulmak için hiyerarşik kümeleme gibi başka bir algoritma türü kullanılır. Hoca bunu bir sonraki derste anlatacağını söyledi. Plan şöyleydi:

- 4. bölüm, gözetimsiz öğrenme, hiyerarşik kümelemeyle bitirilecek.
- Ardından 6. bölüme, Information Retrieval and Web Search'e geçilecek. Bu, web içerik madenciliği (web content mining) bölümüdür.
- Sonra web yapı madenciliği (web structure mining), yani ağ analizi ve bağlantı analizi (link analysis) gelecek.
- Ardından web taraması (web crawling), yani verinin nasıl tarandığı, ve web kullanım madenciliği (web usage mining) işlenecek.
- Vakit kalırsa görüş madenciliği (opinion mining) ve duygu analizi (sentiment analysis) de anlatılacak.

Hoca, bu dört beş konunun hepsinin ilişkilendirme kuralı madenciliği, sıralı örüntü madenciliği, gözetimli ve gözetimsiz öğrenme tekniklerini kullandığını söyledi. Bu teknikler ve başka bazı algoritmalar web verisine uygulanır.

23.1 Proje İçin Fikir Örnekleri

Hoca, şimdiye kadar öğrenilen temel algoritmaların farklı web madenciliği kategorilerinde nasıl kullanılabileceğine dair örnekler verdi:

- Belirli bir tarama algoritmasına dayanarak web'den veri indiren bir tarayıcı (crawler) yazmak.
- Web'den çekilen belge kümeleri üzerinde kümeleme yapmak.
- Kullanıcı verisinden öğrenen bir gözetimli öğrenme uygulaması geliştirmek.
- Hibrit bir arama uygulaması: Sayfa arka planda farklı e-ticaret sitelerinden, örneğin televizyonla ilgili bütün ürünleri sürekli tarar. Ürünler karşılaştırmalı bir liste olarak gösterilir. Her ürünün yanında hangi sitede satıldığı ve fiyatı yazar. Ürüne tıklanınca ilgili sayfaya gidilir.
- İnternette hazır bulunan web veri kümeleri üzerinde bölümlerde anlatılan türden uygulamalar geliştirmek.

Hoca önceki yıllardan bir öğrenci projesini de örnek gösterdi:

1. Dinamik URL'ler toplandı ve her URL için etiket bulundu. Etiketleme önce elle yapıldı.
2. URL'ler parçalarına ayrıldı.
3. Parçalar açık kaynak bir word2vec kütüphanesiyle vektörlere dönüştürüldü. Her dinamik URL bir vektörle temsil edildi.
4. Vektörlere bölümlemeli bir kümeleme algoritması ve temsilci bir hiyerarşik kümeleme algoritması, farklı uzaklık ölçüleriyle uygulandı.
5. Ortaya çıkan kümeler görselleştirildi ve başarımları ölçüldü.

Hoca, son sınıf öğrencilerinin yaptığı bu çalışmanın iyi bir makaleye dönüştüğünü söyledi.

Örnek Sorular

Soru 1 (derste sözlü, 00:19:57 ve 00:21:17) $\langle\{3\}\{4,5\}\{8\}\rangle$ dizisinin boyutu ve uzunluğu nedir?

> **Çözüm (derste verildi):** Boyut eleman sayısıdır: $\{3\}$, $\{4,5\}$, $\{8\}$, yani 3. Uzunluk toplam öğe sayısıdır: $1+2+1=4$. Dizi bir 4-dizidir.

Soru 2 (derste sözlü, 00:24:15) $s_1 = \langle a_1 a_2 \ldots a_r \rangle$'nin s_2'nin alt dizisi olması için $a_1, a_2, a_3, \ldots$ elemanlarının s_2'nin elemanlarına sıralı bir şekilde mi alt küme olması gerekir?

> **Çözüm (derste verildi):** Evet. a_1, s_2'nin bir elemanının alt kümesi olmalı; a_2, ondan sonra gelen ayrı bir elemanın alt kümesi olmalı; böyle devam eder. Elemanlar eşit de olabilir, önemli olan kapsamadır.

Soru 3 (derste sözlü, 00:24:58) s_1'in s_2'nin alt dizisi olması için boyutlarının eşit olması gerekir mi?

> **Çözüm (derste verilen cevap düzeltildi):** Hoca "evet" dedi, ama doğrusu hayırdır. Tanımda $r \leq v$'dir. Örneğin boyutu 3 olan $\langle\{3\}\{4,5\}\{8\}\rangle$, boyutu 5 olan $\langle\{6\}\{3,7\}\{9\}\{4,5,8\}\{3,8\}\rangle$'nin alt dizisidir ($\{3\} \subseteq \{3,7\}$, $\{4,5\} \subseteq \{4,5,8\}$, $\{8\} \subseteq \{3,8\}$). Python ile denetlendi.

Soru 4 (derste sözlü, 00:45:08) Kredi verisinde hiç öğrenme yapmadan yeni bir başvurunun etiketi hangi doğrulukla tahmin edilebilir?

> **Çözüm (derste verildi):** İki sınıf olduğu için hoca önce %50 dedi. Ardından çoğunluk sınıfını kullanarak hesapladı: 15 öğenin 9'u Yes olduğu için her öğeye "Yes" denirse doğruluk $9/15 = \%60$ olur. Doğru taban çizgisi %60'tır. Bir öğrenme algoritması bundan daha iyi olmalıdır.

Soru 5 (derste sözlü, 01:20:45) Naive Bayes'te $P(C_1)$ ve $P(\text{age} \leq 30 \mid \text{yes})$ nasıl hesaplanır?

> **Çözüm (derste verildi):** 14 öğenin 9'u buys_computer = yes olduğu için $P(C_1) = 9/14$, $P(C_2) = 5/14$. Yes olan 9 öğenin 2'sinde age ≤ 30 olduğu için $P(\text{age} \leq 30 \mid \text{yes}) = 2/9$.

Soru 6 (derste sözlü, 02:27:28) Kümeleme yaparken elde edilecek kümelerin eliptik olup olmadığını nasıl anlarız?

> **Çözüm (derste verildi):** Önceden bilemeyiz. Farklı kategorilerden temsilci algoritmalar (k-means, bisecting k-means, agglomerative kümeleme) ve farklı uzaklık ölçüleri denenir. Sonuçlar karşılaştırılarak en iyi kümeleme seçilir.

Dersin Toparlaması

Bu derste önce sıralı örüntü madenciliği tamamlandı. Dizi, eleman, boyut, uzunluk ve alt dizi kavramlarıyla, işlem verisinin müşteri dizilerine nasıl dönüştürüldüğü görüldü. Sık dizileri bulan GSP algoritmasının Apriori ile aynı iskelete sahip olduğu gösterildi: tara, say, aday üret, ele, tekrarla. Bu yöntemin web'de tıklama akışı verisinden sık gezinme örüntüleri bulmak için kullanıldığı anlatıldı.

Ardından gözetimli öğrenmenin temel kavramları (eğitim ve test verisi, doğruluk, öğrenmenin tanımı, çoğunluk sınıfı taban çizgisi) ve dört yöntem işlendi: modeli bir ağaç olan karar ağacı, modeli bir kural kümesi olan sınıf ilişkilendirme kurallarıyla sınıflandırma, modeli bir olasılıklar kümesi olan Naive Bayes ve model kurmayan kNN. Son bölümde gözetimsiz öğrenmeye geçildi. Kümelemenin kullanım alanları, küme kalitesi ölçüleri, k-means algoritması, durma ölçütleri, disk üzerinde çalışması, $O(tkn)$ karmaşıklığı, zayıf yönleri ve bunların çözümleri ile küme gösterimi anlatıldı. k-means'in yalnız yuvarlak ya da eliptik kümeleri bulabilmesi, bir sonraki derste işlenecek hiyerarşik kümelemeye geçişi hazırladı. Ardından 6. bölümle web içerik madenciliğine geçilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.